05 — Backend Schema and Data Contracts

Product: EventPilot — The Living Plan **Status:** Logical schema; adapt to the chosen framework and storage after repository inspection.

1. Storage strategy

Default MVP: browser-local state if allowed by challenge rules. Do not add database or authentication solely for appearance. If cloud persistence is required, choose a single appropriate service and document access rules before implementing it.

2. Entities

EventPlan

- `id`: string/UUID
- `eventType`: `wedding | birthday | community`
- `title`: string
- `eventDate`: ISO date (`YYYY-MM-DD`)
- `guestCount`: non-negative integer within an explicit safe bound
- `currency`: valid supported currency code
- `totalBudgetMinor`: non-negative safe integer
- `priorities`: array of short strings
- `budgetItems`: `BudgetItem[]`
- `tasks`: `EventTask[]`
- `ideas`: `Idea[]`
- `createdAt`, `updatedAt`: ISO timestamps

BudgetItem

- `id`: string
- `category`, `label`: non-empty bounded strings
- `amountMinor`: non-negative safe integer
- `included`: boolean
- `provenance`: `user-entered | ai-estimate | confirmed-by-user`
- `note`: optional bounded string
- `locked`: boolean

EventTask

- `id`: string
- `title`, `category`: bounded strings
- `status`: `todo | in-progress | done | blocked`
- `dueDate`: ISO date
- `dependsOn`: array of task IDs
- `priority`: `low | medium | high`
- `locked`: boolean

- `provenance`: `user-entered | ai-estimate | confirmed-by-user`

Idea

- `id`, `title`, `description`, `rationale`: bounded strings
- `estimatedCostMinor`: optional non-negative safe integer
- `provenance`: `ai-estimate`

PlanChange

- `id`: string
- `entityType`: `event | budget-item | task | idea`
- `entityId`: string
- `field`: string from an allowlist
- `before`, `after`: typed values
- `reason`: bounded string
- `requiresApproval`: always true for user-visible proposed changes
- `status`: `pending | accepted | rejected`

3. Validation

- Reject negative, non-finite, or out-of-range monetary values and guest counts.
- Validate event and task dates strictly as calendar dates.
- Reject unknown enum values and unexpected fields at API boundaries.
- Bound string lengths, list sizes and request payload sizes.
- Validate all dependency IDs and reject cycles.
- Do not trust values supplied by the browser simply because the UI validates them.
- Validate AI response schema and domain constraints before storing/rendering.

4. AI response contract (illustrative)

```
{
  "ideas": [
    {
      "title": "string",
      "description": "string",
      "rationale": "string",
      "estimatedCostMinor": 0,
      "assumptions": ["string"]
    }
  ],
  "suggestedTaskChanges": [
    {
      "taskId": "existing-task-id",
      "proposedDueDate": "YYYY-MM-DD",
      "reason": "string"
    }
  ]
}
```

This is a proposed application-level contract, not a claim about a specific Gemini API schema. Map it to the current supported structured-output implementation after checking official documentation.

5. State transitions

- Draft plan -> editable plan.
- Editable plan + constraint change -> pending change set.
- Pending change -> accepted or rejected per item.
- Accepted changes -> committed plan with new updated timestamp.
- Rejected changes -> original plan retained.
- Locked items cannot be automatically mutated.
- AI failure -> no destructive state transition.

6. Error contract

Return safe structured errors:

- `INVALID_INPUT`
- `AI_UNAVAILABLE`
- `AI_RATE_LIMITED`
- `AI_TIMEOUT`
- `AI_INVALID_RESPONSE`
- `STORAGE_UNAVAILABLE`
- `DEPENDENCY_CYCLE`

Errors must contain a user-safe message and, where useful, a recovery action. Do not expose secrets, stack traces or raw provider details to end users.

7. Privacy

Store only what is necessary. Avoid collecting names, phone numbers, addresses or vendor contact details unless a confirmed requirement demands them. Provide clear reset/delete behavior. If local storage is used, explain that data is kept in the current browser.